SE(3)-WAM: Learning Dynamic Manipulation from Forecasts at Action Time

Anonymous CVPR submission

Figure placeholder: Fig. 1 teaser — learning to act on the future 4D scene (real robot)

Layout: full width, two parts. **(a) Left, ∼62%:** one real-robot frame of the *conveyor grasp* at the instant a chunk is issued. Overlays in the paper's semantic colors: blue = the target's past SE(3) poses from the causal tracker (small triads at $-350 \ldots 0$ ms, fading with age); orange = target SE(3) triads at the commands' application times, the one at the first application time τ^{a} emphasized, taken from the training-time world head evaluated offline; light orange arrows = coarse 3D flow on the context on the 14×14 grid only; purple = the commanded gripper path meeting the target at τ^{a}. A gray ghost marks the pose observed at capture, with a dashed red arrow showing where a reactive policy would close. A thin timeline strip under the image: capture $\tau_t \rightarrow$ latency $\ell_t \rightarrow$ application τ^{a}. **(b) Right, ∼38%:** top: "Image-space WAM: future frames or latents" (filmstrip icon); middle: "SE(3)-WAM: SE(3) for the target, 3D flow for the context, indexed by application time" (triads + sparse arrows); bottom **(required)**: SR vs. added latency (0–200 ms) for SE(3)-WAM and the strongest baseline (data: the latency sweep of Appendix G.2). **Rules:** real photo kept unretouched; overlays drawn from logged tracker/forecast/command data of that trial; caption states the trial's outcome.

Figure 1. **Forecasting the scene at action time.** Between image capture and the first command, a part on the conveyor moves farther than the grasp tolerates. SE(3)-WAM encodes the target's metric motion (blue) and is trained to forecast the target's SE(3) pose at each command's application time, together with a coarse 3D flow of the surrounding scene (orange, predicted only during training), so that its commands meet the target there (purple). Image-space world–action models forecast appearance instead. Right: success against added latency on Dynamic-14.

Abstract

Manipulating moving objects requires commands that fit the scene at the moment they take effect, a few hundred milliseconds after the image was captured. We analyze this delay with an error budget in physical time, in which the target's contact error grows with its speed times the lead, while the rest of the scene only needs to be predicted to within its clearance. The budget suggests what a policy should remember (the target's metric motion), what it should predict and how precisely (an SE(3) trajectory for the target and a coarse 3D flow for the context), and for which instants (the commands' application times). We build SE(3)-WAM, a world–action transformer that follows these choices and learns lead-conditioned commands together with this 4D forecast. On Dynamic-14, fourteen dynamic tasks that we evaluate in real time, SE(3)-WAM succeeds in [XX.X]% of held-out episodes against [XX.X]% for the strongest baseline. Supervising the forecast at the commands' application times gives the benefit of prediction without generating a forecast at test time, so that deployment runs only the action stream. A base pretrained on DROID with automatically lifted 4D labels transfers to a real conveyor grasp, where SE(3)-WAM succeeds in [XX]% of 180 trials against [XX]% for $\pi_{0.5}$.

1. Introduction

Picking a part from a conveyor, placing an object into a moving bin, and catching a ball that rolls off a ramp all require a command that fits the scene at the moment it takes effect. Between image capture and gripper closure, the policy runs inference, commands wait in a queue, and the arm moves. A part on a conveyor at 0.10 m/s travels 15 mm during a 150 ms delay, which is more than the [X] mm that a parallel-jaw grasp of it tolerates. Most state-of-the-art visuomotor policies, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [10, 29, 39, 46], were developed for quasi-static scenes, where this delay hardly matters [1, 12, 43].

Recent work adds temporal context in three ways. Policies receive stacked frames, optical flow, or compressed video tokens as history [22, 26, 28, 43, 56], world models forecast future video, latents, object states, or velocity fields [12, 33, 46, 52], and execution schemes keep commands timely despite inference delay [6, 34, 37]. It remains unclear which information a dynamic policy needs, how precisely, and for which instant. We argue that the answer follows from where interaction errors come from. The error of a grasp or a placement is set by the target's metric pose at the moment the command applies, and the rest of the scene matters only through the clearance it leaves. Pixel histories leave this motion implicit, and image-space forecasts spend

most of their capacity on appearance.

We make this argument concrete with an error budget in physical time (Sec. 3). It splits the contact error into localization, motion, timing, unmodeled-dynamics, and rotation terms, and the rest of the scene matters only where its prediction error exceeds the clearance. The algebra is elementary and latency compensation is classical in control and tracking [3, 4, 32], but for learned policies the budget gives a way to allocate precision across the scene and predictions that we can test, such as the lead beyond which a policy that ignores motion must fail. It also suggests three design choices. A policy should *remember* the target's metric motion, *predict* the target's SE(3) trajectory and a 3D flow of the context no finer than its clearance requires, and *act* on forecasts and commands indexed by their application time.

We build SE(3)-WAM around these choices (Fig. 3). A transformer encodes the current RGB-D frame, the robot state, and the target's SE(3) history from a causal pose tracker. A world stream predicts the target's SE(3) trajectory and a coarse 3D scene flow at physical horizons, and an action stream generates commands that are conditioned on their lead, the time from image capture to their application. The world stream supervises the shared representation during training, and deployment runs the action stream alone. For the real robot, we lift DROID [19] to metric 4D labels with off-the-shelf perception [16, 17, 30] and pretrain a base model on it.

We evaluate on Dynamic-14, fourteen simulated tasks that we built so that each term of the budget can be varied on its own, on DOMINO [12] and LIBERO [24], and on a real conveyor, in a real-time protocol in which the scene keeps moving while each method computes. SE(3)-WAM reaches [XX.X]% success on held-out Dynamic-14 scenes, [X.X] points above the strongest baseline, although a larger model is ahead when physics pauses during inference. The SE(3) history helps beyond the critical lead, whereas RGB, flow, or video-latent histories add little. Most of this gain survives when the forecast is dropped at deployment, as long as it was supervised at the times the commands apply, much as [48] found for static scenes. The budget's predictions hold approximately, with the largest deviations on the capture tasks[, and one task remains unsolved by every method]. In summary, we contribute

- a simple error budget in physical time that motivates the design and whose predictions we test,
- SE(3)-WAM, which learns lead-conditioned actions together with a role-dependent 4D forecast, and a real-robot base pretrained on DROID with lifted 4D labels, and
- Dynamic-14 with a real-time protocol, on which SE(3)-WAM improves over prior methods. We will release it with the DROID labels and our code.

2. Related Work

Dynamic manipulation. Compensating delay by prediction is classical in control [32] and predictive tracking [3]. Robotic catching and hand–eye systems estimate an object's trajectory from vision, usually with Kalman-type filters [4], and intercept it [2, 5, 14]. Visual servoing closes the loop on image features [18], and dynamic grasping couples motion prediction with reachability [1]. Among learned policies, DOMINO introduces a dynamic benchmark together with PUMA, which reads historical flow and is supervised with future object features [12]. DynamicVLA streams actions and drops stale ones [43], DynamicWAM conditions a WAM on flow and kinematic descriptors [28], and others predict salient future tokens [33] or event timing [53]. Several recent preprints pursue related ideas with motion-aware world models, velocity fields, or synthesized dynamic demonstrations [9, 21, 49]. For execution, real-time chunking keeps control smooth under delay [6], VLASH rolls the robot state forward to the execution time [34], and the concurrent Chronos Policy conditions on observation and execution timestamps [37]. These methods handle time on the robot side or learn object motion from pixels, whereas we ask which scene quantities a policy must know at execution time.

World–action models. WAMs couple future prediction with action generation through predictive features [15], visual subgoals [54], or joint video–action generation [20, 39, 46, 58]. Others add depth, semantics, or dynamic regions to the forecast (e.g., [47, 52]) or compress long histories [26]. Fast-WAM finds that test-time imagination can be skipped without loss on its static tasks [48]. These models forecast in image or latent space, without the metric, object-level state that the error budget points to. SE(3)-WAM forecasts such a state at physical horizons, and we find that in dynamic scenes, too, its benefit survives without test-time imagination when the forecast is supervised at the commands' application times.

Structured 3D and 4D motion. Rigid-body structure has long been used in learned dynamics models and scene flow [7, 8, 35, 45], and 3D point tracking follows pixels through space [42]. Point tracks and 3D flow have served as policy interfaces [41, 44, 57], point clouds as policy inputs [50], and object poses as structure for world models and policies [11, 27]. PointWorld predicts action-conditioned 3D point flow at scale [16], and feed-forward models reconstruct scene geometry [38] and dynamic point tracks [51]. We use rigid structure only for the target and keep the rest of the scene as a coarse 3D flow, both indexed by the commands' application times.

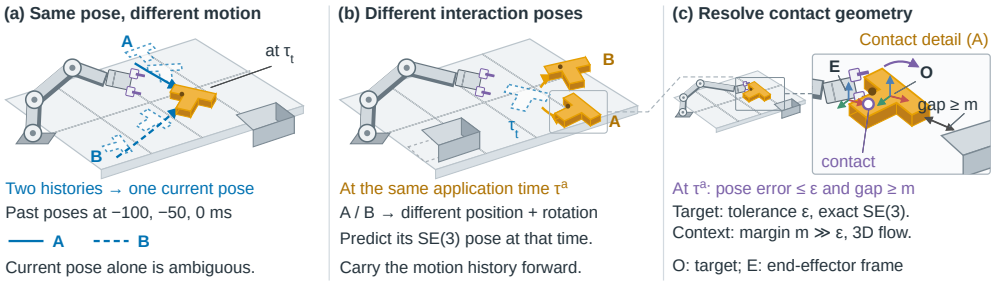


Figure 2. **Three consequences of acting in physical time** (schematic). (a) Two histories reach the same pose at capture time τ_t but imply different motions. (b) At the same application time τ^{a}, they lead to different target poses, in translation and in rotation. (c) A contact requires the end-effector frame E to be aligned with the target O within ε, whereas the context matters only through the gap it leaves, with a margin $m \gg \varepsilon$.

3. Interaction in Physical Time

Physical time. We consider a policy that receives images and robot states together with their capture times. At query t, its history is $\mathcal{H}_t = \{(o_i, s_i, \tau_i)\}_{\tau_i \le \tau_t}$, and it outputs a chunk $A_t = (\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1})$ whose j-th command takes effect at

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + \ell_t + j\Delta_a, \tag{1}$$

where ℓ_t is the latency from capture to the first application (preprocessing, inference, queuing, and actuation) and Δ_a is the control period. We call $\Delta_{t,j} = \tau^{\mathrm{a}}_{t,j} - \tau_t$ the *lead* of command j, which must fit the scene at $\tau^{\mathrm{a}}_{t,j}$ rather than at τ_t.

Target and context. Let $W_t(\mathbf{x}, \tau) \in \mathbb{R}^3$ be the displacement between τ_t and τ of the scene point that is at $\mathbf{x}$ at time τ_t, expressed in the robot base frame. We split the scene by the precision that the task requires of it (Fig. 2). The *target* $\mathcal{O}$ is the rigid body whose pose error enters a contact or containment tolerance ε, e.g., an object to grasp or a receptacle to place into. The *context* $\mathcal{C}$ contains the bodies that only need to be avoided—a belt, a wire, an occluder—with clearance m. On the target, the motion field is set by its pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$:

$$W_t(\mathbf{x}, \tau) = T(\tau)\, T(\tau_t)^{-1}\mathbf{x} - \mathbf{x}, \qquad \mathbf{x} \in \mathcal{O}. \tag{2}$$

An interaction error budget. Consider a contact point at lever arm r from the target origin and a policy that acts, explicitly or implicitly, on an estimate $\hat{T}$ of the target pose at τ^{a}. With a rotation error θ, the contact error satisfies

$$e_{\mathrm{c}} \le \|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\| + 2r\sin(\theta/2). \tag{3}$$

We write the true motion over the lead $\Delta = \tau^{\mathrm{a}} - \tau_t$ as $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$, where $\boldsymbol{\rho}$ is what constant-velocity extrapolation misses (acceleration, rolling, impacts), and the estimate as $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Adding and subtracting $\mathbf{v}\hat{\Delta}$ gives

$$\|\hat{\mathbf{p}} - \mathbf{p}\| \le \underbrace{e_{\mathrm{loc}}}_{\text{now}} + \underbrace{\hat{\Delta}\, e_v}_{\text{lead}\times\text{motion}} + \underbrace{\|\mathbf{v}\|\, e_\tau}_{\text{speed}\times\text{timing}} + \underbrace{e_{\mathrm{dyn}}}_{\text{dynamics}}, \tag{4}$$

with $e_{\mathrm{loc}} = \|\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\|$, $e_v = \|\hat{\mathbf{v}} - \mathbf{v}\|$, $e_\tau = |\hat{\Delta} - \Delta|$, and $e_{\mathrm{dyn}} = \|\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}\|$. Rotation decomposes in the same way (Appendix A). The context enters differently. If the commanded path keeps a clearance m from the predicted context positions $\hat{\mathbf{y}}_t(\mathbf{x}, \tau)$, a collision requires a prediction error of at least m at some point and time in the set $\mathcal{P}$ that the path passes:

$$\text{collision} \;\Rightarrow\; \max_{(\mathbf{x},\tau)\in\mathcal{P}} \big\|\hat{\mathbf{y}}_t(\mathbf{x}, \tau) - \mathbf{y}_t(\mathbf{x}, \tau)\big\| \ge m. \tag{5}$$

A trial succeeds if $e_{\mathrm{c}} \le \varepsilon$ and no collision occurs. Eqs. (3)–(5) follow from an identity and the triangle inequality. We use them as a budget rather than as a model of success.

Predictions. Given each task's tolerance and speeds (Appendix B.1): (i) a policy that ignores motion incurs an error of $\|\mathbf{v}\|\Delta$, so it should fail beyond the critical lead $\ell^\star = \varepsilon/\|\mathbf{v}\|$, and its success should depend on the task, speed, and latency only through $\|\mathbf{v}\|\Delta/\varepsilon$. (ii) A least-squares fit to n poses with noise σ over a span T estimates the velocity with an error of about $\sigma\sqrt{12/n}/T$ [4], so a pose history remains useful up to a noise level of $\sigma^\star = \varepsilon T\sqrt{n}/(\Delta\sqrt{12})$. (iii) A forecast that is misaligned with the application times by δ adds $\|\mathbf{v}\|\delta$, which grows with speed and vanishes for stationary targets. (iv) A context forecast with path error e_F is sufficient as long as $m > e_F$.

Design implications. Table 1 maps each term to the part of a policy that can reduce it. *Remember.* Given a reliable current pose, a history reduces e_v and, through acceleration, part of e_{dyn}. Because e_v is multiplied by the lead, the history matters through the target's metric motion state, and pixels, flow, or video tokens help only if a policy recovers this state

Table 1. **Predictions of the error budget.** Each row concerns a term of Eqs. (4)–(5), and Sec. 5.3 tests it on held-out Dynamic-14 scenes by sweeping the named factor. $\ell^\star$ and $\sigma^\star$ are the critical lead and noise level, and e_F is the path error of the context flow.

Principle	Term	Prediction
All	e_c/ε	Without compensation, success depends on task, speed, and latency only through $\|\mathbf{v}\|\Delta/\varepsilon$ and drops once it exceeds about 1.
Remember	$\hat{\Delta}e_v$	SE(3) history helps beyond $\ell^\star$ ([X]–[X] ms across tasks), increasingly with latency and speed.
	$e_v(\sigma)$	Its gain persists up to pose noise $\sigma^\star$ ([X]–[X] mm).
Predict	e_{dyn}, $r\theta$	Learned SE(3) futures beat extrapolation on non-constant motion and point tracks under rotation.
	m	Coarse flow suffices down to clearance e_F.
Act	$\|\mathbf{v}\|e_\tau$	A 50 ms horizon shift costs 5 mm at 0.10 m/s and nothing for static targets. Leads in control steps fail when the latency changes.

from them. *Predict.* Reducing e_{dyn} requires a model of how the target will move, and rotation errors are amplified by r. The target's future should therefore be a full SE(3) trajectory, which by Eq. (2) determines the motion of all its points. Context errors are tolerated up to m, so a 3D flow with an error below m suffices, and appearance does not enter the budget. *Act.* Every term is evaluated at τ^{a}, which depends on the latency. Forecasts and commands should therefore be indexed by physical time, with each command's lead known to the policy. Whether the action should read a forecast online or be trained to encode it is left open by the budget, and we answer this question experimentally (Sec. 5.3).

4. SE(3)-WAM

SE(3)-WAM (Fig. 3) is a transformer with three groups of tokens. Condition tokens encode the observation and the target's motion history (*Remember*), world queries forecast the motion field at physical horizons (*Predict*), and action tokens generate lead-conditioned commands (*Act*). We predict the world deterministically and generate actions with flow matching [23].

4.1. Remember: Metric Target-Motion History

We estimate the target pose in every RGB-D frame at full resolution with a causal tracker. SAM 2 propagates a mask from a prompt on the target [30], and FoundationPose tracks the object from a CAD model or a few reference views [40]. Without either, we use a monocular 3D detector that estimates a metric box pose in every frame [17]. Each estimate is emitted once, with its capture time and a validity flag, and later frames do not revise it. We resample this causal track at $-350, -300, \ldots, 0$ ms, so that its meaning does not depend on the camera rate, and build 8 motion tokens with interval motions, finite-difference velocities and accelerations, and the current pose, in the base frame and in physical units (Appendix C.1). A frozen MAE ViT-B/16 [13] encodes the current RGB image into 196 patch tokens, and the depth image gives each patch a metric 3D position for rotary attention. Robot tokens hold 8 end-effector states and the pose commanded for the first application time, and the model receives no past images. We train on the same causal pose estimates that the policy sees at test time, tracking failures included.

4.2. Predict: A Role-Dependent Motion Field

The world queries forecast W_t at horizons $\tau_t + \delta_h$, with $\delta_h = h{\cdot}50$ ms and $h = 1, \ldots, 8$. These horizons cover the leads of all commands that are executed before the next replan (Appendix A.4). We parameterize the target through Eq. (2). For each horizon,

$$\mathbf{d}_h = \mathbf{p}(\tau_t{+}\delta_h) - \mathbf{p}(\tau_t), \quad \boldsymbol{\phi}_h = \mathrm{Log}\big(R(\tau_t{+}\delta_h)R(\tau_t)^\top\big)^\vee. \tag{6}$$

An object query regresses $(\mathbf{d}_h, \boldsymbol{\phi}_h)$ with a squared translation error plus λ_R times the squared geodesic rotation error ($\mathcal{L}_P$). For rotationally symmetric targets, we exclude the unobservable rotation about the symmetry axis. We represent the rest of the scene as a 3D scene flow, the base-frame displacements $F_h \in \mathbb{R}^{N\times 3}$ of $N = 196$ points on a fixed 14×14 image grid, which are about [X] mm apart in the workspace. One scene query per point, with an embedding of its role (target, robot, or context), regresses the point's displacements and visibility ($\mathcal{L}_F$). The grid is deliberately coarse, as the budget allows, and Sec. 5.3 tests where this is sufficient. Since appearance does not enter the budget, future depth, DINOv3 features [31], and the RGB of moving surfaces serve only as additional training targets for the scene queries ($\mathcal{L}_{\mathrm{aux}}$). All labels are cumulative from the current state, and losses use valid labels only.

4.3. Act: Lead-Conditioned Actions

A chunk holds 8 commands that are applied at the times of Eq. (1). Each command is a base-frame translation and a left-relative rotation of the end effector, plus a gripper command. We take increments relative to the pose commanded for the preceding application time, which the controller knows, instead of the arm's pose at capture [34]. Each action token embeds its lead $\Delta_{t,j} = \ell_t + j\Delta_a$. During training, we sample $\ell \sim \mathcal{U}[0, 250\,ms]$ and read the command labels from the demonstration at $\tau_t + \ell + j\Delta_a$, so that a single model serves any latency in this range. For a normalized chunk $\mathbf{a}_1$, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and $u \sim \mathcal{U}[0, 1]$, the action loss is

$$\mathcal{L}_A = \mathbb{E}\left\|v_\psi(\mathbf{a}_u, u \mid \mathcal{Z}_t, \hat{\mathcal{W}}_t, \mathcal{G}) - (\mathbf{a}_1 - \mathbf{a}_0)\right\|^2, \tag{7}$$

with $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$, condition tokens $\mathcal{Z}_t$, and the world $\hat{\mathcal{W}}_t$ to the extent that the coupling exposes it. The total loss is $\mathcal{L} = \mathcal{L}_A + \lambda_F\mathcal{L}_F + \lambda_P\mathcal{L}_P + \lambda_{\mathrm{aux}}\mathcal{L}_{\mathrm{aux}}$.

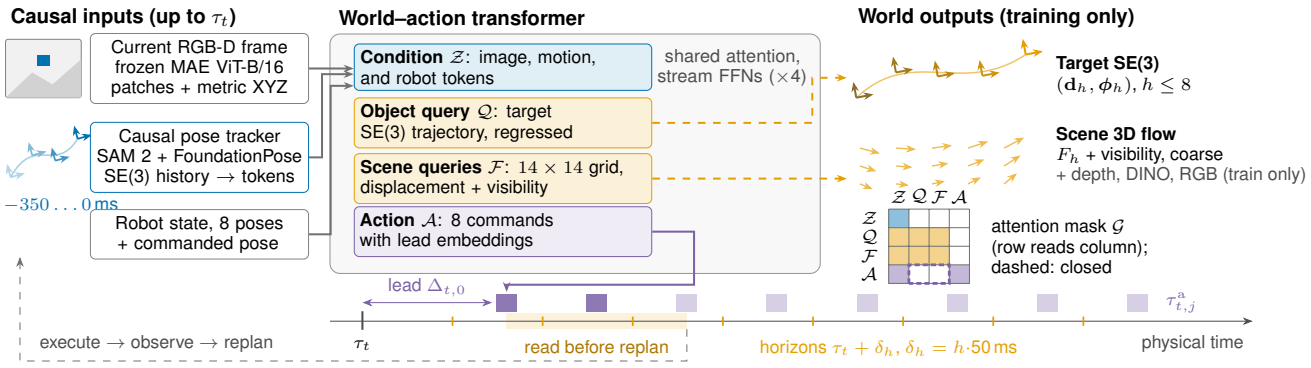


Figure 3. **SE(3)-WAM.** Condition tokens $\mathcal{Z}$ encode the current RGB-D frame, the target's SE(3) history from a causal tracker, and the robot state (*Remember*). An object query $\mathcal{Q}$ and scene queries $\mathcal{F}$ regress the target's SE(3) trajectory and a coarse 3D scene flow with visibility at physical horizons $\tau_t + \delta_h$, and appearance heads add training targets (*Predict*). Action tokens $\mathcal{A}$ generate lead-conditioned commands that apply at $\tau_{t,j}^{a}$, with replanning every 100 ms (*Act*). The mask $\mathcal{G}$ sets which groups each group attends to. Action tokens attend to no world token, so deployment runs the action stream alone.

The attention mask $\mathcal{G}$ acts on four token groups: the condition $\mathcal{Z}$, the object query $\mathcal{Q}$, the scene queries $\mathcal{F}$, and the actions $\mathcal{A}$. Condition tokens attend only to each other, world queries attend to the condition and to each other, and action tokens attend to the condition, to themselves, and to the part of the world that their coupling allows. In Sec. 5.3, we compare couplings in which the action reads nothing from the world, the hidden states of the world queries, or the decoded geometry, and one in which the world also reads the action. In SE(3)-WAM, the action reads no world token. The world losses shape the shared parameters during training, and deployment runs only the action stream. The two streams share the attention projections and have separate feed-forward networks.

Execution and implementation. At test time, we set the leads from the executor's latency budget, the recent p95 inference time plus the actuation delay. The transformer has 4 layers of width 768 with 8 heads, 47.3M backbone parameters, and 62.2M trainable parameters in total. We keep the image encoder and the DINOv3 teacher frozen. On Dynamic-14, we train for 4 epochs (about 40k updates at a global batch size of 16) on eight RTX PRO 6000 GPUs, which takes about 7.7 hours per training seed. At test time, the action stream integrates 5 Euler steps in half precision, with a p95 time-to-action of [XX] ms on an RTX 5090 (Appendix C.6).

4.4. Pretraining on Real Robot Data Lifted to 4D

Real robot data provide neither object poses nor scene flow, so we lift DROID [19] to metric 4D labels without object models. The scene flow comes from the 3D tracks and optimized camera calibrations that PointWorld released for DROID [16]. We take the object named in the instruction as the target. An open-vocabulary detector finds it [25], SAM 2 tracks its mask, and the monocular 3D detector estimates its metric pose. Inputs come from a forward track, as at deployment, and the labels of Eq. (6) from a smoothed forward–backward track over the whole episode. DROID's targets move mostly when the robot moves them ([X]% of windows show target motion before contact), so pretraining mainly teaches metric geometry, 3D flow, and their coupling with lead-conditioned actions. We initialize the policy of each real-robot task from this base. The simulation model is trained from the generic initialization, and a controlled comparison post-trains the base on Dynamic-14 (Appendix E).

5. Experiments

5.1. Setup

Dynamic-14. We built fourteen tasks in MuJoCo/robosuite [36, 59] in five families: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact, each exercising particular terms of the budget (Appendix B.1, Fig. S1). Sweeps vary the speed (0.5–1.5×), the added latency (0–200 ms), the passage clearance ([values] mm), and the pose noise. Scripted experts with simulator state provide 800 demonstrations per task. We select configurations on development scenes and report every Dynamic-14 number on 1,400 held-out scenes (100 per task, new seeds). Our variants average three training seeds, and each baseline is trained once. We report seed means and give paired 95% intervals from a bootstrap over held-out scenes.

Protocol. All methods run in real time under the same executor, and physics advances while a method computes, by its measured latency on one RTX 5090. A new chunk is requested every 100 ms, or when the previous inference

ends, commands execute at their application times, and stale commands are dropped. The +150 ms setting adds 150 ms of delay for every method, and Appendix G.2 compares this protocol with three alternatives. We report the task-balanced success rate (SR), the contact-time error e_t, the position error at first contact e_p, and the p95 time-to-action. On DOMINO [12], we evaluate single-arm interception and tracking, and LIBERO [24] checks that the structured world does not cost performance on static tasks.

Baselines. We compare with reactive policies (ACT [55], Diffusion Policy [10], and $\pi_{0.5}$ [29]), dynamic VLAs (DynamicVLA [43] and PUMA [12]), image- or latent-space WAMs (DreamVLA [52] and OpenWAM-α [39]), and a dynamic WAM (DynamicWAM [28]). Each baseline uses its official code, inputs, and recipe, is trained once on the same demonstrations, and receives a rendered wrist camera if its recipe uses one (Appendix F). SE(3)-WAM uses only the agent view, but it also uses depth and the tracked target pose. To separate these inputs from the representation, $\pi_{0.5}$ and DynamicWAM also receive our SE(3) history tokens (+SE(3)), and we report SE(3)-WAM with poses from the monocular 3D detector, which needs no object model. Two baselines use the tracker alone. *CV input* is SE(3)-WAM without world supervision that receives the constant-velocity extrapolation of the tracked pose as input tokens, and a Kalman interceptor plans to the target pose predicted by a constant-acceleration Kalman filter [4].

5.2. Main Results

In real time, SE(3)-WAM reaches [XX.X]% SR on held-out Dynamic-14 scenes (Table 2), [X.X] points above the strongest baseline, [name of the strongest baseline], and [X.X] points above the strongest baseline that receives the same SE(3) history, [name of the strongest +SE(3) baseline]. With physics paused during inference, however, [name of the strongest paused baseline] leads by [X.X] points. Without the cost of computation, its larger model and richer forecasts pay off, but a query takes it [XXX] ms. The margin of SE(3)-WAM is largest on [rotating targets ($r\theta$) and on fast conveyor and translating targets, where $\|\mathbf{v}\|\Delta$ dominates][, and no method solves the SE(3) placement into the rotating receptacle]. With 150 ms of added delay, SE(3)-WAM keeps [XX.X]%, whereas reactive policies drop to [XX.X]% and image-space WAMs to [XX.X]%. Detector poses instead of a CAD model cost [X.X] points. The CV-input variant and the Kalman interceptor trail SE(3)-WAM by [X.X] and [X.X] points, mostly on non-constant motion, and giving SE(3)-WAM the extrapolated pose as an extra input changes SR by [$\pm$X.X] points (Appendix D.3). On DOMINO, SE(3)-WAM obtains [XX.X]% against [XX.X]% for [name of the strongest DOMINO baseline]. On static LIBERO tasks, it trails $\pi_{0.5}$ by [X.X] points ([XX.X]% vs. [XX.X]%).

5.3. Testing the Predictions

We test each prediction of Table 1 with variants of SE(3)-WAM that change one design axis and keep the data, the training budget, and the other axes fixed (Table 3, Fig. 4, and Appendix D).

Uncompensated motion. We plot the success of every method against $\|\mathbf{v}\|\Delta/\varepsilon$, the target's motion during the lead relative to the tolerance (Fig. 4a). Across tasks, speeds, and latencies, the reactive policies [fall on one curve] (Spearman $\rho = [X.XX]$), and their success halves at [X.X]. The budget gives only a threshold of about one, not the shape of this curve. [The capture tasks scatter most around the curve], because their success also depends on when the gripper closes. Methods that estimate the target's motion shift the curve to the right, SE(3)-WAM the most.

Remember. We hold the current frame, the current target pose, and the robot history fixed. Adding eight frames of RGB, optical flow over the same window with DynamicWAM's motion statistics [28], or video-VAE latents changes SR by [X.X] points (95% interval [[a, b]]). The causal SE(3) history adds [X.X] points and the oracle history [X.X] (Table 3). The gain appears beyond each family's critical lead, grows with latency, and vanishes for stationary targets (Fig. 4b). Frozen readouts show that visual histories carry the motion (Appendix D.2), but at our data scale the policy does not turn it into metric velocity, even with metric XYZ for past frames ([$\pm$X.X] points). Under injected pose noise, the SE(3) history falls to the level of the best visual history at [X] mm (Fig. 4c). This is [X.X] times the model-free prediction $\sigma^\star$, as a learned estimator with a motion prior can be expected to do better, and [X] times the tracker's error.

Predict. Supervising the target's SE(3) trajectory instead of point tracks changes SR by [$\pm$X.X] points overall but adds [X.X] on rotating targets, where the lever arm amplifies rotation errors. Fitting SE(3) to the predicted tracks recovers [X.X] of them. At the application times, the learned SE(3) forecast is as accurate as constant-velocity and constant-acceleration extrapolation of the same history on conveyor motion, but [X] times more accurate on rolling, sliding, and post-impact motion, where e_{dyn} dominates (Fig. 4d). For the context, the 14×14 flow, with a path error of $e_F = [X]$ mm, matches 28^2 and 56^2 grids until the passage clearance falls to [X] mm (Fig. 4e), at [X] times lower training cost for the world stream. Removing the context flow costs [X.X] points, mostly on passage tasks. As the predicted world, future RGB, VAE latents, depth, DINO features, or latents of a frozen video model bring no gain over the SE(3) trajectory and flow ([$\pm$X.X] points at matched capacity), at up to [X] times the

Table 2. **Main results.** Held-out SR (%) on Dynamic-14 (1,400 new scenes, task-balanced, real time, three training seeds for our variants and one for baselines) by family, with 150 ms of added delay, and with physics paused during inference, on single-arm DOMINO and LIBERO (four suites), and p95 time-to-action on one RTX 5090. Inputs: R, RGB and robot state. D, depth. P, target SE(3) history from our tracker with an object model. P′, from a monocular 3D detector. Best in bold. Intervals in Appendix G.

		Dynamic-14, real time unless noted										
Method	Inputs	All	Grasp	Place	Pass	Capture	Impact	+150 ms	Paused	DOMINO	LIBERO	p95 (ms)
ACT [55]	R	—	—	—	—	—	—	—	—	—	—	—
Diffusion Policy [10]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ [29]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
DynamicVLA [43]	R	—	—	—	—	—	—	—	—	—	—	—
PUMA [12]	R	—	—	—	—	—	—	—	—	—	—	—
DreamVLA [52]	R	—	—	—	—	—	—	—	—	—	—	—
OpenWAM-α [39]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM [28]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, CV input	RD+P	—	—	—	—	—	—	—	—	—	—	—
Kalman interceptor	D+P	—	—	—	—	—	—	—	—	–	–	—
SE(3)-WAM, detector poses	RD+P′	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM	RD+P	—	—	—	—	—	—	—	—	—	—	—

Figure placeholder: Fig. 4 — measured trends against the predictions (six panels)

(a) Uncompensated motion. x: $\|\mathbf{v}\|\Delta/\varepsilon$ at the first command's application time (log); y: SR. One point per method × family × speed × latency; reactive policies (gray) with a fitted logistic curve and Spearman ρ, the predicted break at 1 marked; motion-compensating baselines and SE(3)-WAM (orange) as shifted curves. **(b) Remember / lead.** x: added latency (0–200 ms); y: ΔSR vs. current frame. Lines: RGB window, optical flow, video-VAE latent (gray), SE(3) causal (blue), detector SE(3) (blue dotted), SE(3) oracle (blue dashed); stationary targets as a faint line; ticks at the families' $\ell^\star$. **(c) Remember / noise.** x: injected pose noise σ (mm; symlog axis with $\sigma = 0$); y: SR of SE(3) history; band: best visual history; vertical markers: predicted $\sigma^\star$ and the tracker's measured error. **(d) Predict / target dynamics.** Bars by motion type (constant velocity, harmonic, rolling/sliding, post-impact): forecast error at τ^a of CV, CA, and the learned SE(3) forecast (orange), with SR of the corresponding policies as dots. **(e) Predict / context.** Passage tasks only. x: clearance (mm); y: SR; lines: no context flow, 14^2, 28^2, 56^2 grids, future latent; markers: each grid's measured e_F. **(f) Act / coupling.** x: added latency; y: ΔSR vs. training-only world supervision (R1); lines: hidden-state access (R2), bidirectional (R4), decoded geometry (R5), world losses off (R0, dotted); stationary targets as points at the right. 95% paired bootstrap bands, held-out scenes, three seeds; one color per family across panels.

Figure 4. **Measured trends against the predictions of the budget.** (a) The success of reactive policies collapses onto one curve of the uncompensated motion relative to the tolerance, and motion compensation shifts this curve. (b) The SE(3) history helps beyond the critical lead and more with latency, while visual histories stay flat. (c) Its value persists up to the predicted noise level $\sigma^\star$. (d) Learned SE(3) forecasts beat extrapolation on non-constant motion. (e) A coarse context flow suffices until the clearance reaches its error. (f) Training-only world supervision matches online use of the forecast at every latency. Bands show 95% paired bootstrap intervals.

training cost. As additional training targets, however, depth, DINO features, and moving-region RGB add [X.X] points.

Act. Indexing in physical time matters as predicted. Shifting the horizons by 50 ms lowers SR by [X.X] points on the fastest third of the scenes and by [X.X] points for stationary targets, and counting leads in control steps from the first command costs [X.X] points when the test latency differs from the training latency. Training-only world supervision with action-only deployment performs on par with the couplings that read the world ([±X.X] points at 0 ms and [±X.X] at 150 ms, Fig. 4f) at [X.X] times lower time-to-action. Disabling the world losses costs [X.X] points, and this gap grows with the lead and vanishes for stationary targets. The benefit therefore comes from training lead-conditioned actions jointly with forecasts of the scene at their application times.

5.4. Ablations and Pretraining

Removing the SE(3) history, the world losses, or the lead embedding lowers Dynamic-14 SR by [X.X], [X.X], and [X.X] points, and by [X.X], [X.X], and [X.X] under 150 ms of added delay (Table 4). On the real robot, the DROID base raises SR by [X.X] points over the generic initialization (Table 5). On Dynamic-14, post-training the base changes SR by [X.X] points with all demonstrations and by [X.X] with 10% of them, and a base pretrained with actions only

Table 3. **Controlled design comparisons** on held-out scenes (three training seeds). ⋆ marks the configuration selected on development scenes, and each block varies one axis. SR, SR_{150}: success (%) without and with 150 ms of added delay. e_t: contact-time error (ms). e_p: position error at first contact (mm). p95: time-to-action (ms).

Variant	SR	SR_{150}	e_t	e_p	p95
Remember (current frame, current pose, robot history fixed)					
Current frame only	—	—	—	—	—
+ RGB window (8 frames)	—	—	—	—	—
+ optical flow, motion statistics	—	—	—	—	—
+ video-VAE latent	—	—	—	—	—
+ SE(3) history, causal⋆	—	—	—	—	—
+ SE(3) history, oracle	—	—	—	—	—
Predict (future representation)					
No future	—	—	—	—	—
Target tracks + flow	—	—	—	—	—
Target tracks → SE(3) fit + flow	—	—	—	—	—
Target SE(3) only	—	—	—	—	—
SE(3) + 14 × 14 flow⋆	—	—	—	—	—
SE(3) + 28 × 28 flow	—	—	—	—	—
SE(3) + future RGB/latent	—	—	—	—	—
Act (world–action coupling)					
World as training target only⋆	—	—	—	—	—
Action reads world states	—	—	—	—	—
Bidirectional world states	—	—	—	—	—
Action reads decoded geometry	—	—	—	—	—

Table 4. **Ablations of the final model** on held-out Dynamic-14 scenes and DOMINO (SR, %). SR_{150} adds 150 ms of delay.

Model	SR	SR_{150}	DOMINO
SE(3)-WAM	—	—	—
Removed			
SE(3) history (current pose only)	—	—	—
world losses (R0)	—	—	—
lead embedding (latency unseen)	—	—	—
appearance targets (training only)	—	—	—
Replaced by			
leads in control steps	—	—	—
action reads world states	—	—	—
DROID-base initialization	—	—	—

recovers [XX]% of this low-data gain (Appendix E).

5.5. Real-Robot Experiments

A seven-DoF Franka arm with a parallel-jaw gripper must grasp an object that a belt conveyor carries past it, lift it, and hold it for 0.5 s, observed by a calibrated RGB-D camera (Fig. S4). The three objects differ in how much their orientation matters: a box with unequal sides, a slender can, and a cup with one handle. The belt runs at 0.04, 0.08, or 0.12 m/s. Each method is fine-tuned on the same [300] teleoperated demonstrations with three seeds and runs 180 trials, 20 per object and speed, in three blocks of shared trial keys with one seed per block. SE(3)-WAM succeeds in [XX.X]% of its trials (95% interval [[a, b]]), against [XX.X]% for $\pi_{0.5}$ and [XX.X]% without the DROID base (Table 5). Its advantage over $\pi_{0.5}$ grows from [X.X] points at 0.04 m/s to [X.X] at 0.12 m/s and is largest on [the cup]. Its remaining failures are mostly [late closures after tracking losses under the gripper].

Table 5. **Real-robot conveyor grasp.** Success rate (%) by object and by belt speed (slow, mid, fast: 0.04, 0.08, 0.12 m/s), 60 trials each, and over all 180 trials of a method. Every method is fine-tuned with three seeds, one per block of 60 trials. Binomial 95% intervals are given in Appendix F.3.

	Object			Belt speed			
Method	Box	Can	Cup	Slow	Mid	Fast	All
$\pi_{0.5}$ [29]	—	—	—	—	—	—	—
SE(3)-WAM, no DROID base	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—

6. Conclusion

We asked what a policy needs to know, and for which instant, to act on moving objects. An error budget in physical time points to the target's metric motion, its SE(3) trajectory with a coarse flow of the context, and commands indexed by their application time. SE(3)-WAM follows these choices and, in real time, improves on prior policies in simulation and on a real conveyor. The paused-physics comparison shows where the gain comes from. Large video models are strong policies when time stands still, and they lose this edge once the scene keeps moving while they compute. For world–action models, our results favor supervising the future at the instants when commands apply and keeping the deployed policy fast, over generating the future at test time.

Limitations. The budget assumes a rigid target that moves on its own until contact, so deformable or articulated targets and post-contact dynamics are outside its scope. SE(3)-WAM depends on a pose tracker whose failures propagate to the policy. Poses from a monocular 3D detector remove the need for an object model but cost accuracy, and the regressed target forecast cannot represent multimodal futures, e.g., before an impact. Lifting DROID to 4D labels costs substantial computation. Finally, our tasks have a single target and slow context motion, and the real-robot study covers one task on one platform.

LLM Usage Statement

[To be completed before submission: actual tools used, affected parts of the research and manuscript, and extent of use.]

References

[1] Iretiayo Akinola, Jingxi Xu, Shuran Song, and Peter K. Allen. Dynamic grasping with reachability and motion awareness. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9422–9429, 2021. 1, 2

[2] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[3] Ronald Azuma and Gary Bishop. Improving static and dynamic registration in an optical see-through HMD. In *Proceedings of SIGGRAPH*, pages 197–204, 1994. 2

[4] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001. 2, 3, 6

[5] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[6] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, 2025. 1, 2

[7] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, 2017. 2

[8] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 2

[9] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[10] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 6, 7

[11] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. *arXiv preprint arXiv:2503.11565*, 2025. 2

[12] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026. 1, 2, 6, 7

[13] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 4

[14] B. Hove and Jean-Jacques E. Slotine. Experiments in robotic catching. In *American Control Conference*, 1991. 2

[15] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning*, 2025. 2

[16] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 2, 5

[17] Weikai Huang, Jieyu Zhang, Sijun Li, Taoyang Jia, Jiafei Duan, et al. WildDet3D: Scaling promptable 3D detection in the wild. *arXiv preprint arXiv:2604.08626*, 2026. 2, 4

[18] Seth Hutchinson, Gregory D. Hager, and Peter I. Corke. A tutorial on visual servo control. *IEEE Transactions on Robotics and Automation*, 12(5):651–670, 1996. 2

[19] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 2, 5

[20] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[21] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[22] Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu, Siteng Huang, and Donglin Wang. HiF-VLA: Hindsight, insight and foresight through motion representation for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 1

[23] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 4

[24] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023. 2, 6

[25] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 5

[26] Yushan Liu, Jingjing Fan, Shoujie Li, Yifan Xie, Xiao-Ping Zhang, and Wenbo Ding. PACT-WAM: Predicting actions and visual foresight with compact temporal encoding for robot manipulation. *arXiv preprint arXiv:2602.15882*, 2026. 1, 2

[27] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 2

[28] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 6, 7

[29] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 6, 7, 8

[30] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025. 2, 4

[31] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 4

[32] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[33] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. *arXiv preprint arXiv:2606.02486*, 2026. 1, 2

[34] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 1, 2, 4

[35] Zachary Teed and Jia Deng. RAFT-3D: Scene flow using rigid-motion embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. 2

[36] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5

[37] Changhao Wang, Jaden Clark, and Shuran Song. Chronos policy: Learning robot policies on physical time. *arXiv preprint arXiv:2610.13929*, 2026. 1, 2

[38] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2

[39] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 6, 7

[40] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 4

[41] Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[42] Yuxi Xiao, Qianqian Wang, Shangzhan Zhang, Nan Xue, Sida Peng, Yujun Shen, and Xiaowei Zhou. SpatialTracker: Tracking any 2D pixels in 3D space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 2

[43] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026. 1, 2, 6, 7

[44] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, 2024. 2

[45] Zhenjia Xu, Zhanpeng He, Jiajun Wu, and Shuran Song. Learning 3D dynamic scene representations for robot manipulation. In *Conference on Robot Learning*, 2020. 2

[46] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[47] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 2

[48] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. 2

[49] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. *arXiv preprint arXiv:2607.01938*, 2026. 2

[50] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[51] Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, and Mehdi S. M. Sajjadi. Efficiently reconstructing dynamic scenes one D4RT at a time. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 2

[52] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He

Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 2, 6, 7

[53] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[54] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2

[55] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 6, 7

[56] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025. 1

[57] Hongyan Zhi, Peihao Chen, Siyuan Zhou, Yubo Dong, Quanxi Wu, Lei Han, and Mingkui Tan. 3DFlowAction: Learning cross-embodiment manipulation from 3D flow world model. *arXiv preprint arXiv:2506.06199*, 2025. 2

[58] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[59] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5